TERRAFORM CHALLENGE

Cricbuzz Docker Deployment Project – Documentation

1. Project objective

Deploy the Cricbuzz application on an AWS EC2 Ubuntu server using Terraform, Docker Compose, and Nginx.

Main requirements:

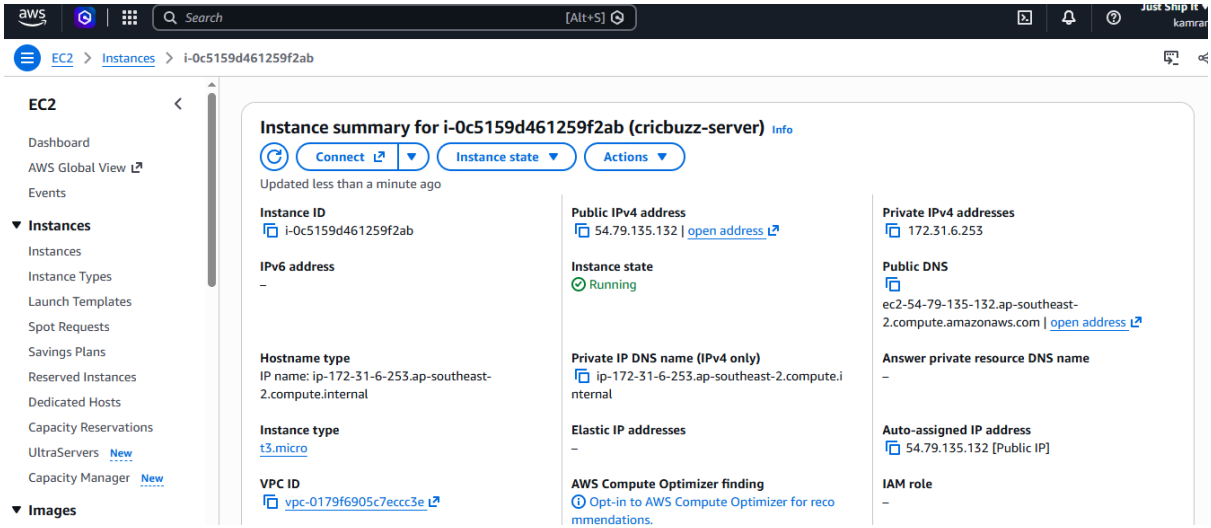
- Provision an Ubuntu EC2 instance using Terraform.
- Use the `t3.micro` instance type.
- Deploy the Cricbuzz application using Docker Compose.
- Run the application internally on port `8000`.
- Configure Nginx as a reverse proxy on port `8080`.
- Store Nginx access logs in `/var/log/cb.log`.
- Configure Logrotate to manage and rotate the access logs.

2. Tools and technologies used

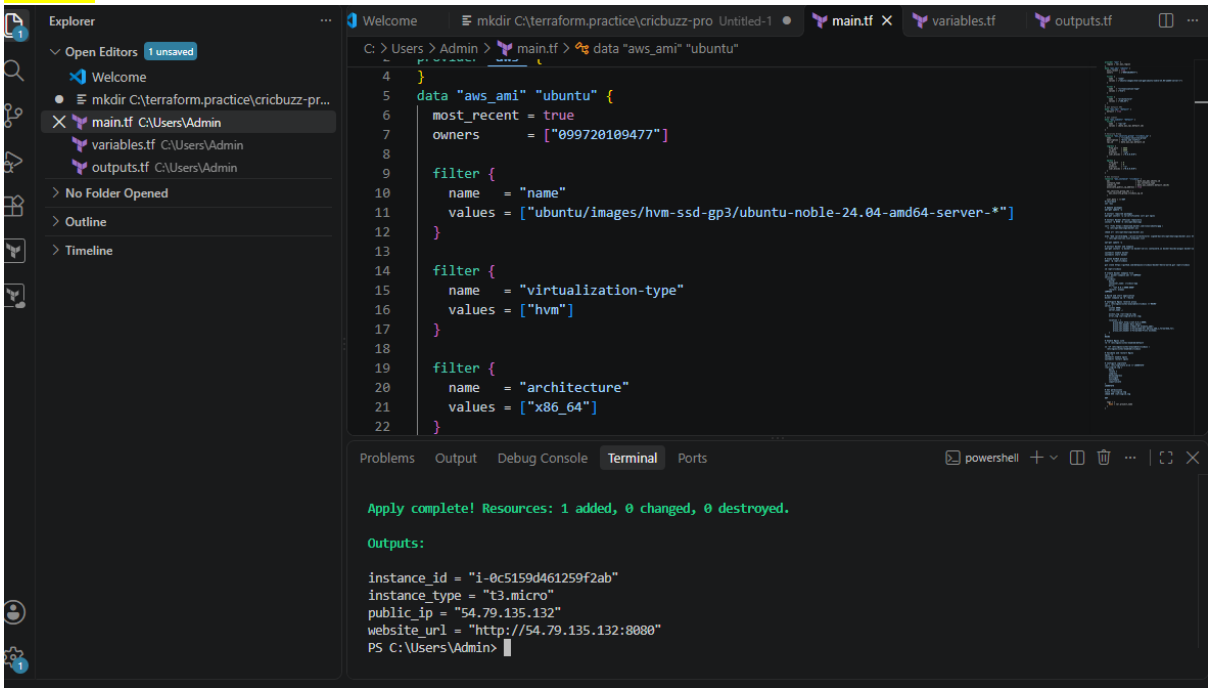
Tool	Purpose
Terraform	AWS infrastructure provisioning
AWS EC2	Host the application
Ubuntu	Operating system
Docker	Containerization
Docker Compose	Application deployment
Go	Application build
Nginx	Reverse proxy and web server
Logrotate	Log management
Git	Clone application source code

Provision EC2 using Terraform

- Created a Terraform project to provision an AWS EC2 instance.
- Configured the AWS provider and selected the required region.
- Used the Ubuntu 24.04 AMI.
- Selected `t3.micro` as the EC2 instance type.
- Configured the default VPC and subnet.
- Created a security group to allow inbound traffic on port `8080`.
- Enabled public IP assignment for external access.
- Used Terraform commands to create the infrastructure.


Search
[Alt+S]
EC2 > Instances > i-0c5159d461259f2ab
EC2
Dashboard
AWS Global View
Events
Instances
Instances
Instance Types
Launch Templates
Spot Requests
Savings Plans
Reserved Instances
Dedicated Hosts
Capacity Reservations
UltraServers New
Capacity Manager New
Images
Instance summary for i-0c5159d461259f2ab (cricbuzz-server) Info
Connect
Instance state
Actions
Updated less than a minute ago
Instance ID
i-0c5159d461259f2ab
Public IPv4 address
54.79.135.132 | open address
Private IPv4 addresses
172.31.6.253
IPv6 address
–
Instance state
Running
Public DNS
ec2-54-79-135-132.ap-southeast-2.compute.amazonaws.com | open address
Hostname type
IP name: ip-172-31-6-253.ap-southeast-2.compute.internal
Private IP DNS name (IPv4 only)
ip-172-31-6-253.ap-southeast-2.compute.internal
Answer private resource DNS name
–
Instance type
t3.micro
Elastic IP addresses
–
Auto-assigned IP address
54.79.135.132 [Public IP]
VPC ID
vpc-0179f6905c7eccc3e
AWS Compute Optimizer finding
Opt-in to AWS Compute Optimizer for recommendations.
IAM role
–

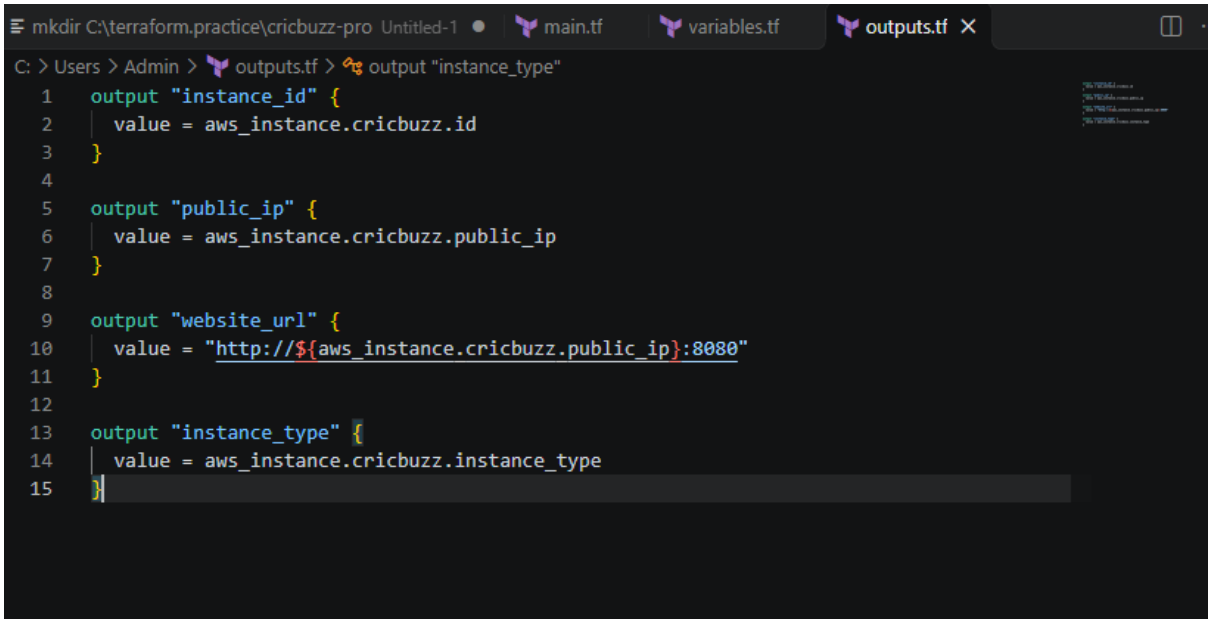
Main.tf


Explorer
Open Editors 1 unsaved
Welcome
mkdir C:\terraform.practice\cricbuzz-pr...
main.tf C:\Users\Admin
variables.tf C:\Users\Admin
outputs.tf C:\Users\Admin
No Folder Opened
Outline
Timeline
Welcome
mkdir C:\terraform.practice\cricbuzz-pro
Untitled-1
main.tf
variables.tf
outputs.tf
C: > Users > Admin > main.tf > data "aws_ami" "ubuntu"
}
data "aws_ami" "ubuntu" {
most_recent = true
owners = ["099720109477"]
filter {
name = "name"
values = ["ubuntu/images/hvm-ssd-gp3/ubuntu-noble-24.04-amd64-server-*"]
}
filter {
name = "virtualization-type"
values = ["hvm"]
}
filter {
name = "architecture"
values = ["x86_64"]
}
Problems
Output
Debug Console
Terminal
Ports
powershell
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
Outputs:
instance_id = "i-0c5159d461259f2ab"
instance_type = "t3.micro"
public_ip = "54.79.135.132"
website_url = "http://54.79.135.132:8080"
PS C:\Users\Admin>

Variables.tf



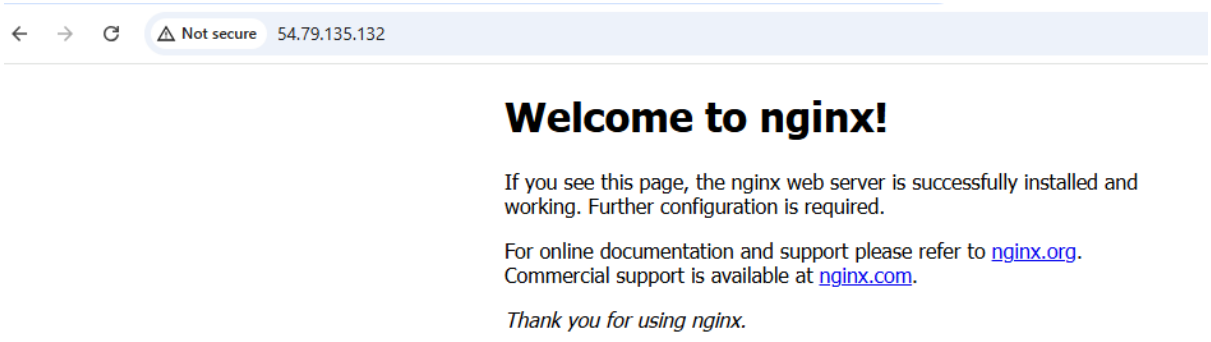
Output.tf



Installed and configure Nginx

- Installed Nginx on the EC2 Ubuntu instance.
- Configured Nginx to listen on port `8080`.
- Configured reverse proxy to forward incoming requests to the Docker application on port `8000`.
- Added request headers to forward client information.

Configured the Nginx access and error log locations.



Installed Docker and Docker Compose

- Installed Docker on the Ubuntu EC2 instance.
- Enabled and started the Docker service.
- Installed or verified the Docker Compose plugin.
- Added the Ubuntu user to the Docker group.

- # Created and configure the Dockerfile

- Reviewed the existing Dockerfile.
- Identified that the expected application binary was missing.
- Updated the Dockerfile to use a multi-stage Go build.
- Used a Go builder image to compile the application.
- Copied the compiled binary into the final image.
- Configured the application to run on port `8080` inside the container.

```
FROM golang:1.20 AS builder


WORKDIR /app


COPY go.mod go.sum ./

RUN go mod download


COPY . .
```

RUN CGO_ENABLED=0 GOOS=linux GOARCH=amd64 go build -o hello_world .

FROM ghcr.io/vincetse/scratch

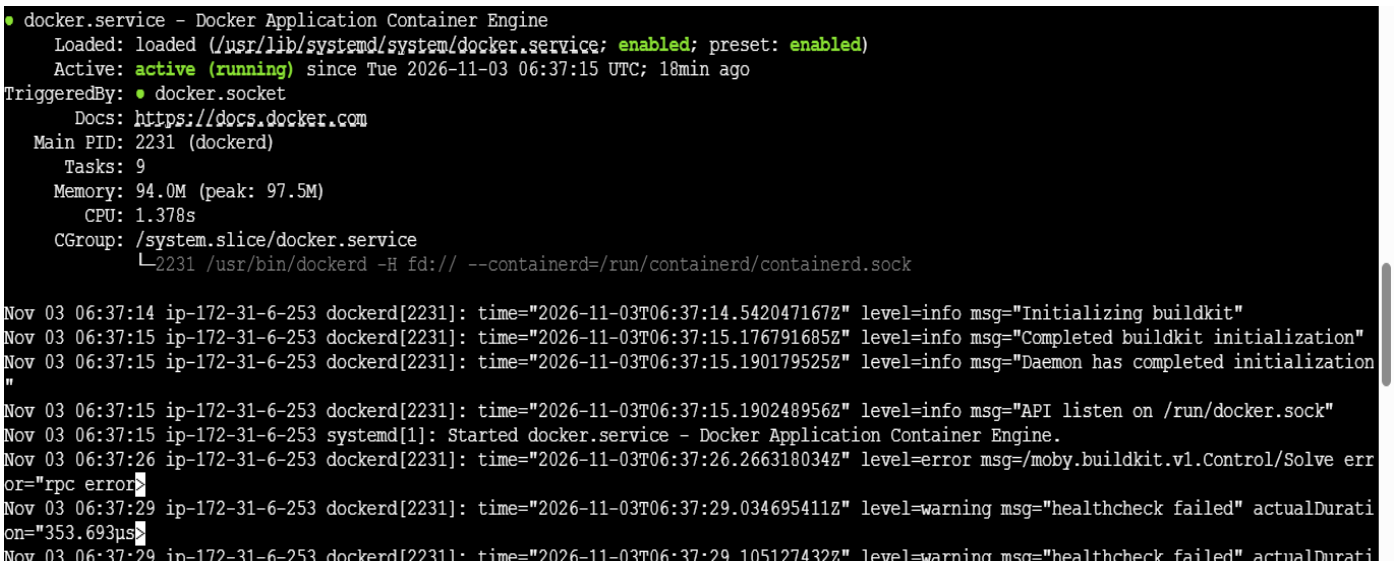
WORKDIR /

COPY --from=builder /app/hello_world .

ENV GIN_MODE=release

EXPOSE 8080

ENTRYPOINT ["/hello_world"]



```
ubuntu@ip-172-31-6-253:/opt/cricbuzz$ cd /opt/cricbuzz
ubuntu@ip-172-31-6-253:/opt/cricbuzz$ ls -la
total 124
drwxr-xr-x 4 root root  4096 Nov  3 06:37 .
drwxr-xr-x 4 root root  4096 Nov  3 06:37 ..
drwxr-xr-x 8 root root  4096 Nov  3 06:37 .git
-rw-r--r-- 1 root root     8 Nov  3 06:37 .gitattributes
drwxr-xr-x 3 root root  4096 Nov  3 06:37 .github
-rw-r--r-- 1 root root   259 Nov  3 06:37 .gitignore
-rw-r--r-- 1 root root   177 Nov  3 06:37 Dockerfile
-rw-r--r-- 1 root root 11357 Nov  3 06:37 LICENSE
-rw-r--r-- 1 root root   142 Nov  3 06:37 Makefile
-rw-r--r-- 1 root root  2329 Nov  3 06:37 README.md
-rw-r--r-- 1 root root   129 Nov  3 06:37 docker-compose.yml
-rw-r--r-- 1 root root  2067 Nov  3 06:37 go.mod
-rw-r--r-- 1 root root 60305 Nov  3 06:37 go.sum
-rw-r--r-- 1 root root  1437 Nov  3 06:37 hello_world.go
-rw-r--r-- 1 root root  1087 Nov  3 06:37 hello_world_test.go
```

Verify Docker container

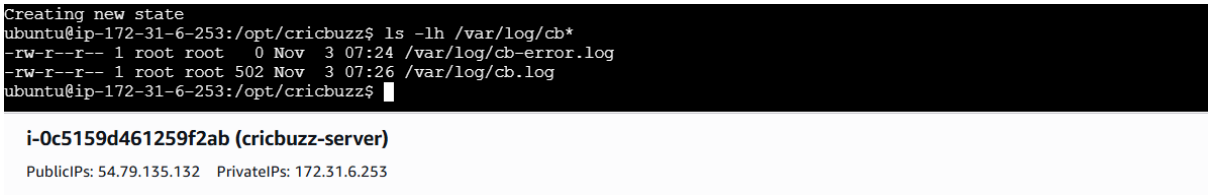
- Checked whether the application container was running.
- Verified the port mapping.
- Tested the application using localhost port `8000`.
- Confirmed that the application returned `Hello, World!`.

```
 => => naming to docker.io/library/cricbuzz-cricbuzz:latest
        0.0s
 => => unpacking to docker.io/library/cricbuzz-cricbuzz:latest
        0.2s
 => resolving provenance for metadata file
        0.1s
[+] up 1/1
[+] up 2/3
 ✓ Image cricbuzz-cricbuzz  Built
        74.1s
 ✓ Network cricbuzz_default Created
        0.1s
 ⠿ Container cricbuzz-app   Creating
 ⠿ Container cricbuzz-app   Creating
 ⠿ Container cricbuzz-app   Starting
[+] up 3/3
 ⠿ Container cricbuzz-app   Starting
 ⠿ Container cricbuzz-app   Starting
 ✓ Container cricbuzz-app   Started
        0.6s
ubuntu@ip-172-31-6-253:/opt/cricbuzz$ sudo docker ps
CONTAINER ID   IMAGE              COMMAND         CREATED         STATUS         PORTS                    NAMES
a17fb4dffccf   cricbuzz-cricbuzz  "/hello_world"  2 minutes ago   Up 2 minutes   127.0.0.1:8000->8080/tcp  cricbuzz-app
ubuntu@ip-172-31-6-253:/opt/cricbuzz$
```

Configure Logrotate

- Created a Logrotate configuration for `/var/log/cb.log`.
- Configured daily log rotation.
- Retained the last seven rotated log files.
- Enabled compression of older logs.
- Used `copytruncate` to truncate the active log after copying it.
- Configuration:
- `/var/log/cb.log {`
- `    daily`
- `    rotate 7`
- `    compress`

- `delaycompress`
- `missingok`
- `notifempty`
- `copytruncate`
- `}`



Browser Access Verification

- Accessed the Cricbuzz application through the EC2 instance's public IP address.
- Used port `8080` to access the application from a web browser.
- Verified that Nginx receives incoming HTTP requests.
- Verified that the application displays the `Hello, World!` response.
- Confirmed external connectivity through the EC2 security group.
- Verified that browser requests are recorded in `/var/log/cb.log`.

